SourceTree 版本控制學習文件（含 GitHub、PR、分支與合併、必備技巧）

適用對象：初學者與進階使用者，使用 SourceTree 圖形介面操作 Git。

一、環境準備與基本設定

- 安裝 SourceTree：至 Atlassian 官方網站下載安裝。
- 安裝 Git：若未安裝，SourceTree 會提示安裝內建 Git 或自訂 Git；建議使用內建即可。
- 建立 GitHub 帳號：登入後於 GitHub 建立個人帳號與個人存取權杖（Personal Access Token，PAT）。
- 在 SourceTree 連結帳號：
  - 開啟 SourceTree → 工具（Tools）→ 選項（Options）→ Authentication。
  - 新增帳號：Host 選 GitHub、Protocol 選 HTTPS，使用 PAT 登入。
- 設定全域使用者資訊（必要）：
  - 工具 → 選項 → Git → 設定使用者名稱與 Email（與 GitHub 帳號一致為佳）。

二、建立與上傳到 GitHub（Push 到遠端）

A. 從零開始建立本機倉庫並上傳

1. 建立本機倉庫：
   - File → New → Create New Repository → 選擇資料夾 → 建立。
2. 新增檔案並 Commit（Check-in）：
   - 在該資料夾加入檔案（例如 README.md）。
   - 回到 SourceTree，於「工作樹（Working Copy）」看到變更 → 勾選檔案 → 輸入 Commit 訊息 → Commit。
3. 建立 GitHub 遠端倉庫：
   - 到 GitHub 建立新 Repository（可選擇是否初始化 README）。
   - 複製遠端 URL（HTTPS）。
4. 在 SourceTree 連結遠端：
   - Repository → Repository Settings → Remotes → Add。
   - Name：`origin`，URL：貼上 GitHub URL → OK。
5. Push 到 GitHub：
   - 點 Push → 選擇分支（通常 `main` 或 `master`）→ Push。

B. 從 GitHub 複製既有專案（Clone）

1. 在 GitHub 複製 HTTPS URL。
2. SourceTree → Clone → 貼上 URL → 選擇本機路徑 → Clone。
3. 完成後可直接在本機開發與 Commit，再 Push 上去。

三、Check-in 與 Check-out 的觀念與操作

- Check-in（在 Git 等價於 Commit + Push）

    - 在本機將變更加入索引（Stage）後 Commit；
    - 需要共享到遠端時，再 Push 到遠端（例如 GitHub）。
- Check-out（在 Git 等價於切換分支或取出特定版本）
    - 在 SourceTree 中切換分支（Checkout）即更新工作目錄到該分支的檔案狀態；
    - 也可使用「檔案歷史」取出某一版（Reset/Checkout file）。

操作步驟

- Check-in：工作樹 → 勾選檔案 → 輸入訊息 → Commit；需要共享 → Push。
- Check-out：左側分支列表 → 右鍵目標分支 → Checkout；或在歷史視圖選擇某一 Commit → Checkout。

四、Pull 與 Fetch（同步遠端變更）

- Fetch：拉取遠端最新引用（refs），不變更本機工作樹；用來查看遠端有無更新。
- Pull：等於 Fetch + Merge/Rebase（將遠端變更套用到目前分支）。

操作

- 點擊工具列 Fetch 按鈕可更新遠端資訊。
- 點 Pull：選擇遠端來源（origin）、分支（如 `main`），合併策略（Merge 或 Rebase），按 Pull。

五、分支（Branching）與合併（Merging）

建立分支（Feature/Hotfix）

1. 在左側分支樹或工具列選 Branch。
2. 命名規則建議：
    - 功能分支：`feature/xxxx`
    - 修補分支：`hotfix/xxxx`
    - 釋出分支：`release/x.y.z`
3. 從 `main` 或 `develop` 建立分支，Checkout 後在新分支上開發。

合併分支（Merge）

1. 確保目標分支（例 `main`）為目前檢出分支（Checkout）。
2. 按 Merge → 選擇來源分支（例 `feature/xxxx`）→ 合併。
3. 如遇衝突（Conflicts），在衝突解決器選擇保留版本或手動編輯，完成後標記已解決並 Commit。

Rebase（進階選項）

- 用於保持線性歷史：在分支上 Rebase 到最新 `main`。
- 操作：選取分支 → 右鍵 Rebase → 選擇目標分支。
- 注意：Rebase 會改寫歷史，推送前請確認團隊政策允許。

六、Pull Request（PR）流程（GitHub）

建立 PR（以功能分支合併到 `main` 為例）

1. 將本機 `feature/xxxx` Push 到遠端。
2. 在 SourceTree 或 GitHub 網頁開啟建立 Pull Request：
   - 比較來源分支：`feature/xxxx` → 目標分支：`main`。
   - 填寫標題與描述、附上變更範圍、測試方式與風險。
3. 指派 Reviewer；等待審核。
4. 若審核要求修改：在同一分支繼續 Commit + Push，PR 會自動更新。
5. 審核通過後 Merge：選擇合併策略（Squash、Rebase 或 Merge commit）。

清理分支

- PR 合併完成後刪除遠端與本機的 `feature/xxxx` 分支，保持倉庫整潔。

七、版本標記（Tagging）與釋出

- 新增標籤：在某一個重要 Commit 右鍵 → Tag；命名如 `v1.0.0`。
- Push Tags：於 Push 視窗勾選「Push tags」。
- GitHub 釋出：在 GitHub 建立 Release，附上變更紀錄與二進位檔（如需要）。

八、常見工作流建議

- GitHub Flow（簡單）：`main` → 建立 `feature/*` → PR → 合併 → 部署。
- Git Flow（複雜）：`main` + `develop`，搭配 `feature/`、`release/`、`hotfix/` 分支；適合多版本維護。
- 依團隊規模與部署頻率選擇；SourceTree 皆支援上述操作。

九、未說明但必須使用技巧（高效與安全）

- 必備 1：設定 `.gitignore`
  - 在專案根目錄建立 `.gitignore`，忽略 IDE 設定、暫存檔、建置輸出等。
  - 在 SourceTree 可直接加入檔案至忽略清單（右鍵 → Ignore）。
- 必備 2：Commit 訊息規範
  - 建議使用約定式提交（Conventional Commits）：`feat: 增加 X 功能`、`fix: 修正 Y 錯誤`。
  - 保持小而清晰的 Commit，易於審查與回溯。
- 必備 3：分支保護與審查規則
  - 在 GitHub 設定 Branch protection：要求 PR 審查、CI 綠燈才能合併；禁止直接 Push 到 `main`。
- 必備 4：避免直接在 `main` 開發
  - 一律建立分支開發；保持 `main` 可部署狀態。
- 必備 5：定期同步（Pull/Rebase）
  - 功能分支定期與 `main` 對齊，減少大型衝突。
- 必備 6：使用 Stash 暫存
  - 當切換分支但有未提交變更時，先使用 Stash 暫存；切換後再套用。
- 必備 7：善用比較與歷史
  - 在 SourceTree 的 File Status/History 比較差異（Diff），審視每次 Commit 的影響。
- 必備 8：使用多遠端（Fork 參與開源）
  - 新增 `upstream` 追蹤原始倉庫；在 Fork（`origin`）開分支、PR 回原倉庫。

- 必備 9：GPG 簽署（進階）
  - 設定 GPG Key，於 Commit/Tag 簽章，提升可信度。

十、問題排除（Troubleshooting）

- Push 認證失敗：檢查 GitHub PAT 是否過期、Scope 是否含 `repo` 權限。
- 遇到大量合併衝突：分治處理，使用內建合併工具逐檔解決；必要時考慮 `rebase` 改線性歷史。
- 誤操作覆蓋：使用 `reflog` 查找歷史（SourceTree 的 Log/History 可視覺化），嘗試回復到對應 Commit。
- 無法 Checkout 分支：確認工作樹乾淨或先 Stash；檢查是否有子模組或 LFS 造成阻礙。

十一、名詞對照與心智模型

- Check-in：在 Git 世界通常指 Commit（到本地）＋ Push（到遠端）。
- Check-out：切換分支或取出某版內容到工作目錄。
- Pull Request：在平台（GitHub）上提出合併請求，用於代碼審查與合併。
- Branch/Merge/Rebase：分支管理與合併策略，建立乾淨可維護的歷史。

十二、每日實務範例（建議流程）

1. 開工：Fetch → Pull `main`。
2. 建分支：`feature/issue-123-add-x` → Checkout。
3. 開發：小步提交（Commit），訊息清楚。
4. 與 `main` 對齊：視情況 Rebase 或 Merge。
5. 推送：Push 分支到 GitHub。
6. 建 PR：填寫描述、指派 Reviewer。
7. 合併：Squash 或 Merge，刪除分支。
8. 發版：打 Tag、建 Release（如需要）。

附註：本文件聚焦 SourceTree 圖形介面操作；對應 Git CLI 指令可作為進階延伸學習。

十三、完整實例教學：從建立到 PR 合併與發版

以下以「建立一個 Java 範例專案，新增 `feature/add-bmi-calculator` 功能並透過 PR 合併到 `main`」為例，逐步示範。

前置準備

- 在 GitHub 建立空白 Repository：`sample-java-app`（不要初始化 README）。
- 準備本機專案資料夾：例如 `c:\projects\sample-java-app`。
- SourceTree 已設定好 GitHub 認證與全域使用者資訊（見第一章）。

步驟 1：在本機建立倉庫並連結 GitHub 遠端

1. SourceTree → File → New → Create New Repository → 選擇 `c:\projects\sample-java-app` → 建立。

2. 在資料夾中加入初始檔案：

- `README.md`（簡要說明）
- `.gitignore`（可使用 GitHub 的 Java 模板：忽略 `target/`、IDE 設定檔等）

3. 回到 SourceTree：在 Working Copy 勾選兩個檔案 → Commit 訊息：`chore: initialize repository with readme and gitignore` → Commit。
4. 連結遠端：Repository → Repository Settings → Remotes → Add → Name `origin`、URL 貼上 GitHub 的 HTTPS URL → OK。
5. Push 到 GitHub：按 Push → 勾選目前分支（`main`/`master`）→ Push。 小技巧：初始 Commit 務必包含 `.gitignore`，避免把 IDE 或建置產物推到遠端。

步驟 2：建立功能分支並開發

1. 在 SourceTree 按 Branch → 命名 `feature/add-bmi-calculator` → 從 `main` 建立並 Checkout。
2. 在專案中新增 Java 程式（示例）：

- `src/main/java/com/example/BMI.java`
- `src/main/java/com/example/BMITester.java`

3. 回到 SourceTree：查看變更 → 勾選檔案 → Commit 訊息：`feat(bmi): add BMI calculator and tester` → Commit。
4. 定期與 `main` 對齊：按 Pull（選擇 Rebase 或 Merge 依團隊政策），避免後續大型衝突。 小技巧：若需要臨時切換分支但尚未完成，請使用 Stash 暫存後再切換，完成後套用 Stash。

步驟 3：推送分支並建立 Pull Request

1. Push 分支：按 Push → 選擇分支 `feature/add-bmi-calculator` → Push。
2. 建立 PR：

- 於 GitHub 開 PR，來源：`feature/add-bmi-calculator` → 目標：`main`。
- 標題：`feat(bmi): add BMI calculator`
- 內容（建議模板）：
  - 變更：新增 BMI 計算與測試類別
  - 測試：手動測試與簡單單元測試（若有）
  - 風險：輸入驗證、邊界值（身高體重）

3. 指派 Reviewer。若被要求修改，直接在該分支繼續 Commit + Push，PR 會自動更新。 小技巧：在 GitHub 設定 Branch Protection，要求至少一位 Reviewer 與 CI 綠燈才能合併，避免錯誤直接進 `main`。

步驟 4：審查通過後合併 PR 並清理分支

1. 選擇合併策略（建議 Squash 以保持乾淨歷史；或依團隊規範）：

- Squash and merge：把多個 Commit 壓成一個，訊息可整理為 `feat(bmi): add BMI calculator`。

2. 合併完成後，刪除遠端分支（GitHub 提供按鈕）與本機分支（SourceTree → 刪除分支）。
3. 在 SourceTree Pull 更新最新 `main`，確保本機同步。

步驟 5：版本標記與釋出

1. 在合併後的該次 Commit 上右鍵 → Tag → `v1.0.0`。
2. Push Tags：Push 視窗勾選「Push tags」。
3. 在 GitHub 建立 Release：名稱 `v1.0.0`，附上變更紀錄（Changelog）。

步驟 6：遇到合併衝突的處理（模擬）

1. 假設同時有人在 `main` 修改了 `BMI.java`，在 PR 合併或 Pull/Rebase 時出現衝突。
2. 在 SourceTree 的合併衝突解決器中：

- 檢視 ours/theirs 差異 → 手動合併 → 標記已解決。

3. 完成後 Commit 以完成合併或 Rebase；再次 Push。

步驟 7：工作日常節奏（套用到本例）

1. 每日開始：Fetch → Pull `main`。
2. 需求來了：從 `main` 開 `feature/add-bmi-calculator` 分支。
3. 小步提交：每個小功能或修正做一次 Commit，訊息規範化。
4. 對齊主線：遇到主線更新時在分支上 Pull（或 Rebase）。
5. 推送與 PR：Push 分支 → 建立 PR → 審查合併。
6. 清理與發版：刪分支、打 Tag、建立 Release。

進一步強化

- 加入簡單 CI：在 GitHub 建立 Workflow（例如 Maven/Gradle 建置與測試），PR 必須通過才可合併。
- 啟用 GPG 簽署：為重要 Commit/Tag 簽章，提升可信度。
- 多遠端協作：Fork 專案，新增 `upstream` 以追蹤原倉庫，定期同步來源更新。

十四、詳細操作步驟實例：Temperature 轉換程式專案（圖解說明）

本章節以建立「Temperature 溫度轉換程式」為實例，提供每個操作的詳細步驟、界面說明與截圖位置提示。

實例情境說明

專案名稱：`temperature-converter`
功能需求：建立一個 Java 程式，提供攝氏與華氏溫度相互轉換功能
開發流程：本機建立 → GitHub 上傳 → 功能分支開發 → Pull Request → 合併 → 版本標記

階段 A：環境設定與帳號連結（首次使用必做）

A-1：安裝與啟動 SourceTree

1. **下載安裝**：
    - 前往 https://www.sourcetreeapp.com/
    - 下載 Windows 版本並執行安裝程式
    - 安裝過程中會提示安裝 Git，選擇「使用內建 Git」或「使用系統 Git」

2. **首次啟動設定**：
    - 啟動 SourceTree
    - 若出現註冊畫面，可選擇使用 Atlassian 帳號或略過
    - 進入主畫面後，會看到空白的 Repository 列表

A-2：設定 GitHub 個人存取權杖（PAT）

1. **在 GitHub 建立 PAT**：
    - 登入 GitHub → 右上角個人圖示 → Settings
    - 左側選單：Developer settings → Personal access tokens → Tokens (classic)
    - 點擊「Generate new token」→「Generate new token (classic)」
    - Note：`SourceTree Access`
    - Expiration：選擇有效期限（建議 90 days）
    - 勾選 Scopes：`repo`（完整存取倉庫）、`workflow`（選用）
    - 點擊「Generate token」→ **立即複製 Token**（離開頁面後無法再查看）
2. **在 SourceTree 加入 GitHub 帳號**：
    - SourceTree 上方選單：Tools → Options
    - 切換到「Authentication」分頁
    - 點擊「Add」按鈕
    - 設定如下：
        - **Hosting Service**：GitHub
        - **Preferred Protocol**：HTTPS
        - **Authentication**：Personal Access Token
        - **Username**：你的 GitHub 使用者名稱
        - **Token**：貼上剛才複製的 PAT
    - 點擊「OK」儲存

A-3：設定全域 Git 使用者資訊

1. **在 SourceTree 設定**：
    - Tools → Options → Git 分頁
    - 設定：
        - **Default user information**：
            - Full Name：`你的姓名`（會出現在 Commit 歷史）
            - Email Address：`your-email@example.com`（建議與 GitHub 相同）
    - 點擊「OK」
2. **驗證設定**：
    - 這些資訊會記錄在每次 Commit 中
    - 可於 Git 歷史查看 Author 資訊

階段 B：建立本機倉庫並上傳到 GitHub

B-1：建立本機 Git 倉庫

1. **建立專案資料夾**：
   - 在 `c:\projects\` 建立 `temperature-converter` 資料夾
2. **在 SourceTree 初始化倉庫**：
   - SourceTree 上方選單：File → New...
   - 選擇「Create Local Repository」
   - 對話框設定：
     - **Destination Path**：瀏覽並選擇 `c:\projects\temperature-converter`
     - **Name**：temperature-converter（自動填入）
     - **Type**：Git
     - 勾選「Also create remote repository」（先不勾選，稍後手動加入）
   - 點擊「Create」
3. **確認初始化成功**：
   - SourceTree 會開啟該倉庫視窗
   - 左側 BRANCHES 區域會顯示 `main`（或 `master`）分支
   - 中間歷史區域為空（尚無 Commit）

B-2：建立初始檔案與 .gitignore

1. **建立 README.md**：
   - 在 `c:\projects\temperature-converter\` 建立 `README.md`
   - 內容：

```
# Temperature Converter

Java 溫度轉換程式（攝氏 ⇔ 華氏）

## 功能
- 攝氏轉華氏：F = C × 9/5 + 32
- 華氏轉攝氏：C = (F - 32) × 5/9
```

2. **建立 .gitignore**：
   - 在同一資料夾建立 `.gitignore` 檔案
   - 內容（Java 專案範例）：

```
# Compiled class file
*.class

# Package Files
*.jar
*.war
```

```
*.ear

# IDE
.idea/
*.iml
.vscode/

# Build
target/
build/
out/

# OS
.DS_Store
Thumbs.db
```

B-3：第一次 Commit（Check-in）

1. **在 SourceTree 查看變更**：
   - 切換到「File Status」分頁（預設應已選取）
   - 下方「Unstaged files」區域會顯示：
     - `README.md`
     - `.gitignore`
2. **Stage 檔案**：
   - 勾選兩個檔案（或點擊「Stage All」按鈕）
   - 檔案會移至「Staged files」區域
3. **輸入 Commit 訊息並提交**：
   - 下方 Commit 訊息區域：
     - 輸入：`chore: initialize project with README and gitignore`
   - 點擊右下角「Commit」按鈕
4. **確認 Commit 成功**：
   - 切換到「History」分頁
   - 會看到第一個 Commit 節點，顯示：
     - Commit 訊息
     - Author（你的名字）
     - Date/Time
     - 分支標籤 `main`

B-4：在 GitHub 建立遠端倉庫

1. **登入 GitHub**：
   - 前往 https://github.com
   - 點擊右上角「+」→「New repository」

2. **設定倉庫**：
    - **Repository name**：`temperature-converter`
    - **Description**（選填）：`Java temperature conversion utility`
    - **Public** 或 **Private**：依需求選擇
    - **重要**：不要勾選「Initialize this repository with a README」（我們已有本機版本）
    - 點擊「Create repository」
3. **複製倉庫 URL**：
    - 建立完成後會顯示快速設定頁面
    - 確保選擇「HTTPS」
    - 複製 URL（格式：`https://github.com/你的帳號/temperature-converter.git`）

B-5：連結遠端倉庫並 Push

1. **在 SourceTree 加入 Remote**：
    - 上方選單：Repository → Repository Settings
    - 切換到「Remotes」分頁
    - 點擊「Add」按鈕
    - 設定：
        - **Remote name**：`origin`（預設名稱）
        - **URL / Path**：貼上剛才複製的 GitHub URL
        - **Username**（選填）：可留空，推送時會自動使用 PAT
    - 點擊「OK」→「OK」
2. **Push 到 GitHub**：
    - 點擊工具列「Push」按鈕（向上箭頭圖示）
    - Push 對話框：
        - **Remote**：origin
        - **Branch**：勾選 `main`（或 `master`）
    - 點擊「Push」
    - 若出現認證視窗，輸入 GitHub 使用者名稱與 PAT
3. **驗證上傳成功**：
    - 前往 GitHub 該倉庫頁面
    - 應可看到 `README.md` 與 `.gitignore`
    - Commit 歷史顯示初始提交

階段 C：建立功能分支並開發

C-1：建立功能分支

1. **在 SourceTree 建立分支**：
    - 確保目前在 `main` 分支（左側 BRANCHES → main 會有勾選標記）
    - 點擊工具列「Branch」按鈕（分岔路圖示）

- Branch 對話框：
  - **New Branch**：`feature/celsius-to-fahrenheit`
  - **Checkout New Branch**：勾選（建立後立即切換）
- 點擊「Create Branch」

2. **確認分支切換成功**：
   - 左側 BRANCHES 區域會新增 `feature/celsius-to-fahrenheit`
   - 該分支前方有 ✓ 勾選標記（表示目前工作分支）
   - 視窗標題列會顯示目前分支名稱

C-2：開發 Temperature.java（第一個功能）

1. **建立 Java 檔案**：
   - 在專案資料夾建立 `Temperature.java`
   - 內容：

```java
public class Temperature {
    /**
     * 攝氏轉華氏
     * @param celsius 攝氏溫度
     * @return 華氏溫度
     */
    public static double celsiusToFahrenheit(double celsius) {
        return celsius * 9.0 / 5.0 + 32.0;
    }

    /**
     * 華氏轉攝氏
     * @param fahrenheit 華氏溫度
     * @return 攝氏溫度
     */
    public static double fahrenheitToCelsius(double fahrenheit) {
        return (fahrenheit - 32.0) * 5.0 / 9.0;
    }
}
```

2. **在 SourceTree Commit 變更**：
   - 切換到「File Status」分頁
   - Unstaged files 顯示 `Temperature.java`
   - 勾選檔案 → Stage
   - Commit 訊息：`feat(temp): add celsius/fahrenheit conversion methods`
   - 點擊「Commit」

C-3：開發測試程式（第二個 Commit）

1. **建立 TemperatureTester.java**：

- 內容：

```java
public class TemperatureTester {
    public static void main(String[] args) {
        // 測試攝氏轉華氏
        double c = 0.0;
        double f = Temperature.celsiusToFahrenheit(c);
        System.out.println(c + "°C = " + f + "°F");  // 應輸出 32.0

        // 測試華氏轉攝氏
        double f2 = 98.6;
        double c2 = Temperature.fahrenheitToCelsius(f2);
        System.out.println(f2 + "°F = " + c2 + "°C");  // 應輸出 37.0
    }
}
```

2. **Commit**：
   - Stage 檔案
   - Commit 訊息：`test(temp): add tester for temperature conversion`
   - Commit
3. **查看分支歷史**：
   - 切換到「History」分頁
   - 會看到：
     - 最新：`test(temp): add tester...`（feature/celsius-to-fahrenheit 標籤）
     - 其次：`feat(temp): add celsius/fahrenheit...`
     - 最舊：`chore: initialize project...`（main 標籤）

階段 D：同步與 Push 分支

D-1：與 main 對齊（模擬多人協作）

情境說明：開發期間，其他人可能已更新 `main`，我們需要先同步。

1. **Fetch 遠端更新**：
   - 點擊工具列「Fetch」按鈕（下載圖示）
   - 會拉取遠端所有分支的最新引用
   - 如果 `origin/main` 有更新，會在歷史圖中顯示
2. **切換到 main 並 Pull**（若有更新）：
   - 左側 BRANCHES → 右鍵 `main` → Checkout
   - 點擊「Pull」按鈕
   - Pull 對話框：
     - **Remote branch to pull**：origin/main
     - **Options**：選擇「Merge」或「Rebase」（建議 Merge）

    - 點擊「OK」

3. **切回功能分支並 Rebase/Merge**：
    - Checkout 回 `feature/celsius-to-fahrenheit`
    - 右鍵 `main` → 選擇「Merge main into current branch」
    - 若無衝突，會自動完成合併

D-2：Push 功能分支到 GitHub

1. **Push 分支**：
    - 確保目前在 `feature/celsius-to-fahrenheit`
    - 點擊「Push」按鈕
    - Push 對話框：
        - 勾選 `feature/celsius-to-fahrenheit`
    - 點擊「Push」

2. **驗證**：
    - 前往 GitHub 倉庫頁面
    - 點擊分支下拉選單，應可看到 `feature/celsius-to-fahrenheit`

階段 E：建立與處理 Pull Request

E-1：在 GitHub 建立 Pull Request

1. **開啟 PR 建立頁面**：
    - Push 後，GitHub 會顯示黃色提示條：「feature/celsius-to-fahrenheit had recent pushes」
    - 點擊「Compare & pull request」
    - 或手動：切換到該分支 → 點擊「Contribute」→「Open pull request」

2. **填寫 PR 資訊**：
    - **Title**：`feat: add temperature conversion functionality`
    - **Description**（範例）：

```
## 變更內容
- 新增 `Temperature.java`：提供攝氏/華氏轉換方法
- 新增 `TemperatureTester.java`：測試程式

## 測試
- 手動執行 TemperatureTester
- 驗證：0°C = 32°F、98.6°F = 37°C

## 風險評估
- 低風險：純新增功能，不影響既有程式碼
```

- **Reviewers**：指派團隊成員（若為個人練習可略過）
- **Assignees**：指派給自己
- **Labels**：選擇 `enhancement`

3. **建立 PR**：
   - 點擊「Create pull request」
   - PR 頁面會顯示 Commits、Files changed、Checks（若有 CI）

E-2：審查與修改（模擬 Review 意見）

情境：Reviewer 要求新增輸入驗證。

1. **在本機繼續開發**：
   - 確保仍在 `feature/celsius-to-fahrenheit` 分支
   - 修改 `Temperature.java`，加入驗證：

```
public static double celsiusToFahrenheit(double celsius) {
    if (celsius < -273.15) {
        throw new IllegalArgumentException("溫度不可低於絕對零度
(-273.15°C)");
    }
    return celsius * 9.0 / 5.0 + 32.0;
}
```

2. **Commit 並 Push**：
   - Stage 變更
   - Commit 訊息：`fix(temp): add absolute zero validation`
   - Push（選擇同一分支）

3. **PR 自動更新**：
   - 回到 GitHub PR 頁面
   - 會自動新增該 Commit 到 PR
   - Reviewer 可再次審查

E-3：合併 PR

1. **審查通過**：
   - Reviewer 點擊「Approve」
   - 若有 Branch Protection 規則，需滿足條件（如至少 1 個 Approve）

2. **選擇合併策略**：
   - **Merge commit**：保留所有 Commit 歷史（預設）
   - **Squash and merge**：壓縮成單一 Commit（推薦，保持歷史乾淨）
   - **Rebase and merge**：線性歷史

3. **執行合併**：
    - 點擊「Squash and merge」
    - 編輯最終 Commit 訊息：`feat: add temperature conversion functionality (#1)`
    - 點擊「Confirm squash and merge」
4. **刪除遠端分支**：
    - 合併完成後，GitHub 提示「Pull request successfully merged and closed」
    - 點擊「Delete branch」按鈕

階段 F：本機清理與同步

F-1：更新本機 main 分支

1. **Checkout 到 main**：
    - SourceTree 左側 BRANCHES → 右鍵 `main` → Checkout
2. **Pull 最新變更**：
    - 點擊「Pull」按鈕
    - 會拉取合併後的 `main`
3. **查看歷史**：
    - 切換到「History」分頁
    - 會看到 Squash 後的單一 Commit：`feat: add temperature conversion functionality (#1)`

F-2：刪除本機功能分支

1. **刪除分支**：
    - 左側 BRANCHES → 右鍵 `feature/celsius-to-fahrenheit`
    - 選擇「Delete feature/celsius-to-fahrenheit」
    - 確認對話框：點擊「OK」
2. **清理遠端追蹤分支**：
    - Repository → Repository Settings → Remotes
    - 點擊「Prune」按鈕（清理已刪除的遠端分支引用）

階段 G：版本標記與釋出

G-1：建立 Tag

1. **在 History 選擇 Commit**：
    - 切換到「History」分頁

    - 右鍵最新的 `main` Commit（含功能合併）

2. **建立 Tag**：
    - 選擇「Tag...」
    - Tag 對話框：
        - **Tag Name**：`v1.0.0`
        - **Tag Message**（選填）：`Release 1.0.0 - Temperature conversion`
    - 點擊「Add Tag」

3. **Push Tag 到 GitHub**：
    - 點擊「Push」按鈕
    - Push 對話框：
        - 勾選「Push all tags」（或僅勾選 `v1.0.0`）
    - 點擊「Push」

G-2：在 GitHub 建立 Release

1. **前往 Releases 頁面**：
    - GitHub 倉庫 → 右側「Releases」→「Create a new release」

2. **設定 Release**：
    - **Choose a tag**：選擇 `v1.0.0`
    - **Release title**：`v1.0.0 - Temperature Converter First Release`
    - **Description**：

    ```
    ## 新增功能
    - 攝氏轉華氏轉換
    - 華氏轉攝氏轉換
    - 絕對零度輸入驗證

    ## 檔案
    - Temperature.java
    - TemperatureTester.java
    ```

    - 附加檔案（選用）：上傳編譯後的 `.jar` 或原始碼壓縮檔

3. **發布**：
    - 點擊「Publish release」

階段 H：進階技巧實戰

H-1：使用 Stash 暫存未完成工作

情境：正在開發新功能 `feature/kelvin-conversion`，突然需要切換分支修補緊急錯誤。

1. **暫存目前變更**：
    - File Status 顯示未提交的變更
    - 點擊工具列「Stash」按鈕（盒子圖示）
    - Stash 對話框：
        - **Message**：`WIP: kelvin conversion in progress`
    - 點擊「OK」
2. **切換分支處理緊急問題**：
    - Checkout 到其他分支（例如 `main` 或 `hotfix/...`）
    - 工作樹乾淨，可安心操作
3. **完成後套用 Stash**：
    - 切回 `feature/kelvin-conversion`
    - 左側「STASHES」區域 → 右鍵最新 Stash
    - 選擇「Apply Stash」（套用但保留 Stash）或「Pop Stash」（套用並刪除）

H-2：解決合併衝突

情境：兩個分支同時修改 `Temperature.java` 的同一方法。

1. **發現衝突**：
    - Merge 或 Pull 時，SourceTree 顯示「Conflicts」
    - File Status 分頁會標示衝突檔案（黃色驚嘆號）
2. **開啟合併工具**：
    - 右鍵衝突檔案 → 「Resolve Conflicts」→「Launch External Merge Tool」
    - 或直接右鍵 → 「Open in External Editor」手動編輯
3. **解決衝突**：
    - 合併工具顯示三個版本：
        - **Mine**（目前分支）
        - **Theirs**（合併來源）
        - **Base**（共同祖先）
    - 手動選擇保留的程式碼或合併兩者
    - 儲存並關閉
4. **標記已解決**：
    - 回到 SourceTree → 右鍵檔案 → 「Mark Resolved」
    - 所有衝突解決後，點擊「Commit」完成合併

H-3：查看與比較歷史

1. **查看特定 Commit 的變更**：
    - History 分頁 → 點選任一 Commit

    - 下方會顯示該 Commit 的檔案變更列表
    - 點擊檔案可查看 Diff（綠色 +新增、紅色 -刪除）

2. **比較兩個 Commit**：
    - 按住 Ctrl，點選兩個 Commit
    - 右鍵 → 「Diff Selected」
    - 會顯示兩者之間的所有差異

3. **Blame（追蹤程式碼作者）**：
    - File Status → 右鍵檔案 → 「Blame」
    - 或透過選單：Actions → Blame
    - 會顯示每一行的最後修改者與 Commit

階段 I：多遠端協作（Fork 與 Upstream）

I-1：Fork 開源專案

1. **在 GitHub Fork**：
    - 前往目標開源專案（例如 `https://github.com/original/repo`）
    - 點擊右上角「Fork」
    - Fork 會建立到你的帳號：`https://github.com/你的帳號/repo`

2. **Clone Fork 到本機**：
    - SourceTree → Clone → 貼上你 Fork 的 URL
    - Clone 後會自動建立 `origin` 指向你的 Fork

I-2：加入 Upstream Remote

1. **新增 Upstream**：
    - Repository → Repository Settings → Remotes → Add
    - 設定：
        - **Remote name**：`upstream`
        - **URL**：原始專案的 URL（`https://github.com/original/repo.git`）
    - 點擊「OK」

2. **同步 Upstream 更新**：
    - Fetch → 選擇 `upstream`
    - Checkout 到 `main`
    - Merge `upstream/main` into `main`
    - Push 到 `origin`（更新你的 Fork）

3. **建立 PR 回原始專案**：
    - 在你的 Fork 開發功能分支
    - Push 到 `origin`

- 在 GitHub 建立 PR，來源選擇你的 Fork 分支，目標選擇原專案的 `main`

階段 J：常見操作快速參考

每日工作流速查表

操作	SourceTree 步驟	說明
開始工作	Fetch → Pull main	同步遠端最新狀態
建立功能分支	Branch → 輸入名稱 → Create	從 main 建立 feature/xxx
查看變更	File Status 分頁	查看未提交的檔案
暫存檔案	勾選檔案（Stage）	加入索引準備提交
提交變更	輸入訊息 → Commit	記錄到本機倉庫
推送分支	Push → 勾選分支	上傳到 GitHub
切換分支	右鍵分支 → Checkout	更新工作目錄
合併分支	右鍵來源分支 → Merge into...	將變更整合到目前分支
暫存工作	Stash → 輸入訊息	臨時儲存未完成變更
套用暫存	右鍵 Stash → Apply/Pop	恢復暫存的變更
查看歷史	History 分頁	瀏覽 Commit 紀錄
建立標籤	右鍵 Commit → Tag	標記版本里程碑
解決衝突	右鍵衝突檔案 → Resolve	手動合併程式碼

工具列按鈕對照

圖示	名稱	功能
↓	Fetch	拉取遠端引用（不變更本機）
↓+	Pull	拉取並合併遠端變更
↑	Push	推送本機 Commit 到遠端
分岔	Branch	建立新分支
合併	Merge	合併分支
盒子	Stash	暫存未提交變更
標籤	Tag	建立版本標籤

十五、實例總結與檢查清單

完整流程回顧（Temperature Converter 專案）

✅ **環境設定**：SourceTree 安裝 → GitHub PAT 設定 → Git 使用者資訊
✅ **初始化**：建立本機倉庫 → 加入 README & .gitignore → 初始 Commit
✅ **遠端連結**：GitHub 建立倉庫 → 加入 Remote → Push
✅ **功能開發**：建立 feature 分支 → 開發程式 → 分次 Commit
✅ **同步**：Fetch/Pull main → Merge/Rebase 對齊
✅ **PR 流程**：Push 分支 → 建立 PR → 審查 → 修改 → 合併
✅ **清理**：刪除遠端/本機分支 → Pull 更新 main
✅ **發版**：建立 Tag → Push Tag → GitHub Release
✅ **進階技巧**：Stash、衝突解決、Blame、多遠端

學習成果自我檢核

- ☐ 能獨立建立本機倉庫並連結 GitHub
- ☐ 理解 Stage → Commit → Push 的流程
- ☐ 能建立並切換分支
- ☐ 會使用 Stash 暫存未完成工作
- ☐ 能解決簡單的合併衝突
- ☐ 理解 PR 的完整生命週期
- ☐ 會建立 Tag 與 Release
- ☐ 能查看歷史並使用 Diff/Blame
- ☐ 理解 Fork 與 Upstream 協作模式
- ☐ 熟悉 .gitignore 的重要性

下一步建議

1. **實際演練**：按本文建立一個真實專案，完整走過所有流程
2. **團隊協作**：與其他人協作，體驗多人開發的衝突處理
3. **CI/CD 整合**：在 GitHub 建立 Actions Workflow，自動化建置與測試
4. **進階 Git**：學習 Rebase Interactive、Cherry-pick、Reflog 等進階功能
5. **命令列對照**：對照 SourceTree 操作學習對應的 Git CLI 指令，增強理解

附註：本實例文件搭配截圖使用效果更佳。建議初學者按步驟實際操作一遍，遇到問題可回頭查閱對應章節。